CITY UNIVERSITY LONDON

# INM376 / IN3005
# Computer Graphics using OpenGL Path Starting Guide

A key component to the coursework is modelling and rendering a path along which the game/simulation will occur. This guide is designed to help you get started with this, but the path will require additional work after completing this exercise. If you haven't yet completed the Vector Mathematics worksheet and Labs 1, 2, and 3, *you should complete these first* before continuing with this guide, which will build on top of what you've learned in earlier labs, particularly Lab 3. In fact, make a copy of your Lab 3 solution to serve as the starting point for this exercise. Note however, this guide is more challenging than the labs (you're working on your coursework now!) and will require you to think carefully about applying the concepts taught in lecture.

In part 1 of this guide, we will write code to generate a centreline, representing the central path of the path using a Catmull-Rom spline. Next, in part 2 we will implement basic camera movement along this path. In part 3 we will generate triangle-based primitives to form the shape of the path.

## Download a helper class

On the class Moodle page is a helper class called `CCatmullRom`, and consists of a .cpp and an .h file. Use these files to overwrite the matching files in your Lab 3 solution. You'll see this new version of the CatmullRom class has many of the same methods from Lab 3, but a few new methods as well, including:

- `void SetControlPoints();`
- `void ComputeLengthsAlongControlPoints();`
- `void UniformlySampleControlPoints(int numSamples);`
- `bool Sample(float d, glm::vec3 &p, glm::vec3 &up = _dummy_vector);`

The top three of these methods are for needed for computing the Catmull-Rom spline for a varying number of points. In addition, there are six methods you'll be implementing in this guide:

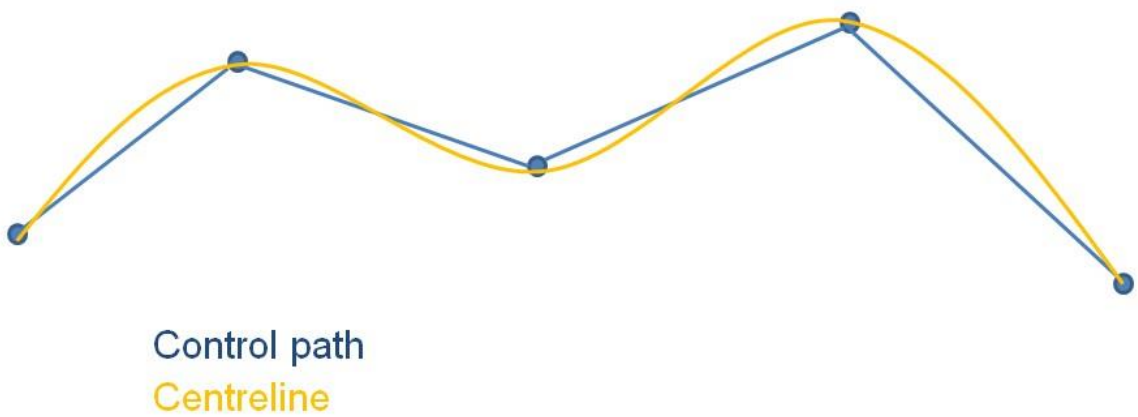
- `void CreateCentreline();`
- `void RenderCentreline();`
- `void CreateOffsetCurves();`
- `void RenderOffsetCurves();`
- `void CreatePath();`
- `void RenderPath();`

The `CreatePath()` and `RenderPath()` methods have been removed however – we're now going to create and render a centreline, then offset curves parallel to the centreline, and finally the path itself.

## Part 1: Centreline modeling

In 3D geometric modelling, a *centreline* is a space curve located in the centre of a geometrical object. We will model this centreline using a Catmull-Rom spline, which you learned in class is continuous and has continuous derivatives (also known as $C^1$ continuity). This spline will serve as the "skeleton" of the path, around which we'll build the path geometry.

In Lab 3 you developed a CatmullRom spline class that uses Catmull-Rom interpolation, which takes four points (**p**0, **p**1, **p**2, and **p**3) to smoothly interpolate the space between the central two points **p**1 and **p**2. This is an effective way to create a smooth path. However, in the coursework we'll want to create a longer path composed of more vertices. As discussed in lecture, we can group sets of four points to create smooth curves between the middle two points. Following the nomenclature of splines, the set of points that are being interpolated are called *control points*. The path the control points take is called the *control path*. Note the control path and the centreline are different, as shown in figure below.




The centreline will interpolate the control path to make a smooth curve. Since the number of control points may vary, we will use a std::vector to store them. In your C++ module taken prior to Computer Graphics, you will have learned about std::vectors, which are part of the standard template library (STL). If you're not familiar with std::vectors, you may wish to review an old but nice and clear vectors tutorial http://www.dreamincode.net/forums/topic/33631-c-vector-tutorial/ and http://www.dreamincode.net/forums/topic/34015-c-vector-tutorial-ii/ before continuing. Note a std::vector is a container (much like an array) in memory to store data, and is different from a mathematical vector like glm::vec3 used to represent points and directions.
Ok, with that in mind, let's create a set of control points and store them as a std::vector. You'll notice in the new `CCatmullRom` class, there is a member variable called `m_controlPoints`, which is a std::vector. In this exercise, to keep things simple we'll use a set of 8 control points –

in your coursework *you should change the path* (and use more points)! Copy the following code to the method `CCatmullRom::SetControlPoints()`

```
m_controlPoints.push_back(glm::vec3(100, 5, 0));
m_controlPoints.push_back(glm::vec3(71, 5, 71));
m_controlPoints.push_back(glm::vec3(0, 5, 100));
m_controlPoints.push_back(glm::vec3(-71, 5, 71));
m_controlPoints.push_back(glm::vec3(-100, 5, 0));
m_controlPoints.push_back(glm::vec3(-71, 5, -71));
m_controlPoints.push_back(glm::vec3(0, 5, -100));
m_controlPoints.push_back(glm::vec3(71, 5, -71));
```

These points are arranged on a circle, and the centreline will loop (end at its beginning). In your coursework, the path should be more interesting (turning left / right, possibly up / down). But for now, this is a good set of points for building the basic functionality.

Next, we would like to use Catmull-Rom spline interpolation to sample a smooth path passing through these points. We'll make the path to have N = 500 points, and will represent the path as a std::vector called `m_centreline`. Fortunately, there is a method already implemented, `CatmullRom::UniformlySampleControlPoints()` that will do this task for us. It takes as an argument the number of samples that should be on the computed centreline path. A nice feature of this method is that it handles the case when the control points are not uniformly spaced, to give a sampled centreline path with roughly equidistant points.

Starting from your Lab 3 solution, in your `Game::Initialise()` method, remove the code that specifies the points and calls `CreatePath()`. Instead, make a call to `m_pCatmullRom->CreateCentreline();`

to create the centreline, and in your `Game::Render()` method, replace the code to render your spline with a call to

```
m_pCatmullRom->RenderCentreline();
```

Implement the `RenderCentreline()` method to render the centreline. I rendered mine twice, one as GL_POINTS and a second time a GL_LINE_LOOP. Note you can retrieve the number of points on the centreline as `m_centrelinePoints.size()`.

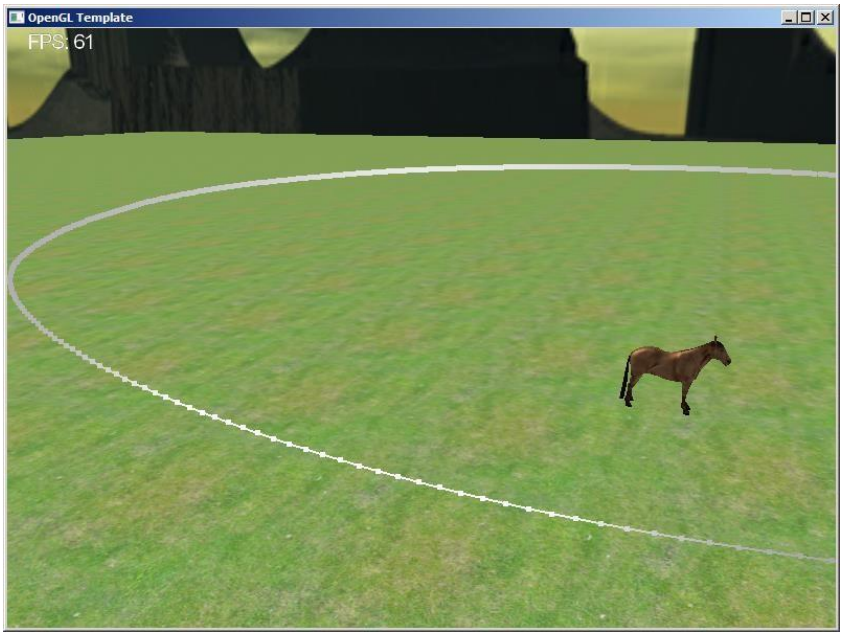
Now in `CCatmullRom::CreateCentreline()`, do the following three things:

1. Call `SetControlPoints()` to get the control points into the `m_controlPoints` std::vector.

2. Call `UniformlySampleControlPoints()` to sample the control points to create points 500 on the centreline. This will generate points along the centreline, and store them in `m_centrelinePoints`.

3. Create a VAO named m_vaoCentreline, and use a VBO to put the centreline points onto the graphics card. You can follow the example from Lab 3 if you're unsure how to do this. Be sure to retrieve each centreline point from the `m_centrelinePoints` vector. For now, set the texture coordinate to (0, 0) and the normal to (0, 1, 0). We'll cover texturing and normals later.

Run your program. If everything is working, you should see the centreline rendered, similar to what we did in Lab 3, only now the path is much larger!




## Part 2: Moving the camera on the centreline

In Lab 3 we moved the camera along the centreline. This was pretty easy because we just sampled the spline to place the camera. However, in Lab 3 we were only moving between two of the control points -- now we have a much larger path to move along.

The new CatmullRom class has a method `CCatmullRom::Sample(float d, glm::vec3 &p, glm::vec3 &up = _dummy_vector)` which takes a positive distance d (along the control path) and sets a corresponding point p on the centreline.

Note, this method has an optional argument up, and can set an up vector if up vectors are provided for the control points in the SetControlPoints() method. This way, one control which way in 3D space the up vector is on the path. However, we'll skip this advanced optional argument for now. We will use `CCatmullRom::Sample()` to calculate the camera position on the spline.

First, add a new floating point member variable in the game class called `m_currentDistance`. Recall you can add a member variable in the Game.h file. This variable will store the distance along the control path we've travelled. In the Game constructor, initialise this variable to 0.0f. Next, in `Game::Update()`, increment the distance by a fixed amount, like `m_dt * 0.1f`. Now in `Game::Update()`, call `m_pCatmullRom->Sample()` to determine a point on the centreline, at a distance of `m_currentDistance`. You can do this by first allocating a glm::vec3 point p and calling `m_pCatmullRom->Sample()`, like this:

```
glm::vec3 p;
m_pCatmullRom->Sample(m_currentDistance, p);
```

Although `m_pCatmullRom->Sample()` can also take an (optional) third argument, you can omit this for now. Place the camera on the centreline at **p**, looking at the origin, with an upvector along the y axis. Run your program. Hopefully you are moving along the spline in a circular orbit, looking at the horse!

Note you can affect the speed of your camera by introducing another floating point variable, like `m_cameraSpeed`, to control the camera speed, which might vary based on game play or user input. You could use `m_cameraSpeed` instead of 0.1f when incrementing the distance the camera has moved along the path.

You should now have an interesting animation, but really what we want is the camera to be looking ahead, *along* the centreline. For this, it is helpful to think of the *TNB frame* (also known as a *Frenet* frame)

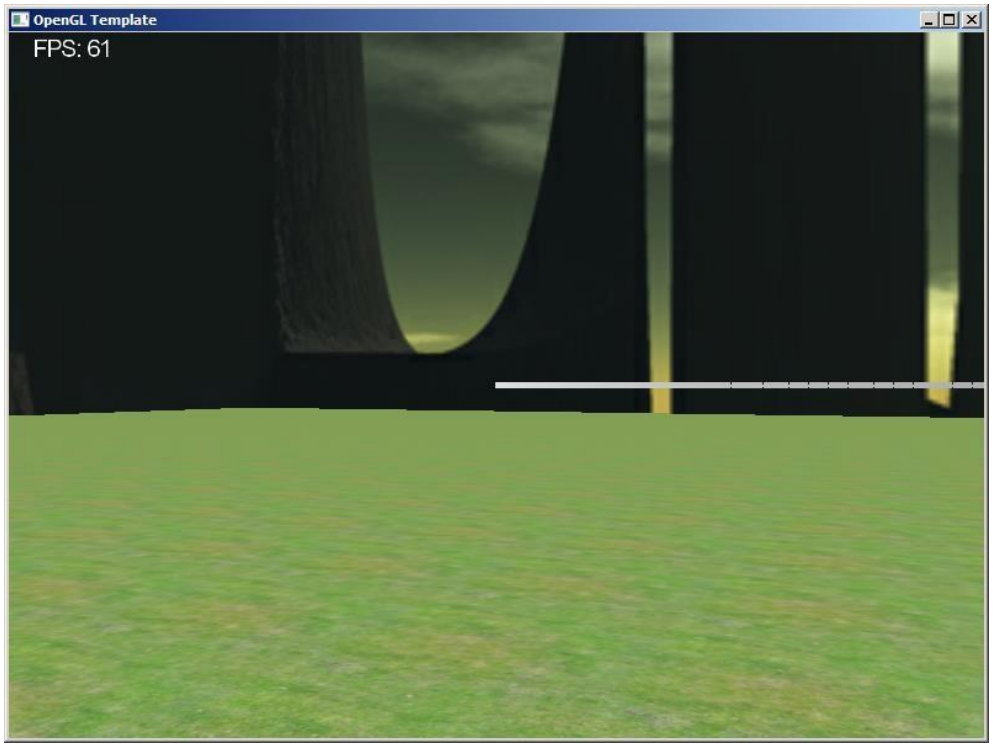
A frame is simply an orthonormal coordinate system, often one that changes along a space curve. The term *orthonormal* means the coordinate system consists of vectors that are mutually orthogonal, and also normalised. The x, y, and z axes of 3D space is an orthonormal coordinate system. However, this coordinate system is fixed (it doesn't change). Instead, we want a coordinate system that is changing as we move along the centreline.

For placing the camera, it is helpful to have a **TNB** frame based on the centreline. **T**, **N**, and **B** stand for three different vectors, which are mutually orthogonal:

- **T**: The tangent vector, which points in the direction of the space curve (i.e., towards the next point on the centreline)
- **N**: The normal vector
- **B**: The binormal vector

The **N** and **B** vectors form a plane that is orthogonal to **T**. With the camera is pointed along the **T** axis, the **N** axis is to the right and the **B** axis is up. In this **TNB** frame, the **N** axis is like the **x** axis, and the **B** axis is like the **y** axis, expressed in camera (eye) coordinates.

First, let's compute the **T** vector and use it to place the camera view point. In `Game::Update()`, before specifying the camera, use the method `m_pCatmullRom->Sample()` to find a point slightly ahead on the spline. For example, you could find the point at (`m_currentDistance` + 1.0f). Call this **p**Next. Using vector mathematics, determine a *normalised* tangent vector **T** that points from **p** to **p**Next. Now set the camera view point to a point that is ahead of **p** in the **T** direction, for example **p** + 10.0f * **T**. Run your program. You now see the camera looking ahead along the path, with the camera rotating as it moves along the spline path! The part of the centreline immediately in front of the camera will be clipped due to the near clipping plane.



You may be wondering about the **N** and **B** vectors. There are different ways to compute these. A simple way is to use cross-products, that is

**N** = **T** x **y**

**B** = **N** x **T**

Where **y** is the **y** axis, $[0, 1, 0]^T$.This should look familiar – in the Vector Mathematics worksheet, question 2, we did a similar operation to determine the **N** vector. Later, if you specify normals for the control path, you can have the m_pCatmullRom->Sample() method return interpolated up vectors (to replace **y**) in the equation above.

Note is customary to normalise all the vectors in the **TNB** frame, make sure yours are normalised!

## Part 3: The path

We would now like to create some path geometry using the centreline. We'll do this using two offset curves, one to the right of the centreline, and one to the left. In your coursework, you're welcome to use more offset curves, for example, to making walls on either side of the path.

## Part 3a: Generating the path vertices

Recall in the Vector Mathematics Worksheet, Question 2, you modelled a path using two offset curves around the centreline. We're going to implement this now.

First, create the path vertices by generating a *left* and a *right* offset curve. Since we have the **TNB** frame, you'll notice the **N** vector is always pointing to the right when we are facing forward on the path. So if we move a positive distance along the **N** vector from a point **p** on the centreline, we'll be at a point to the right of the centreline. Similarly, if we move a negative distance along the **N** vector from **p**, we'll be at a point to the left of the centreline. Let the path width be *w*. We can define the edges of the path as

**l** = **p** - (w/2) * **N**

**r** = **p** + (w/2) * **N** where **l** is the left point, and **r** is the right point, and **p** is

a point on the centreline.

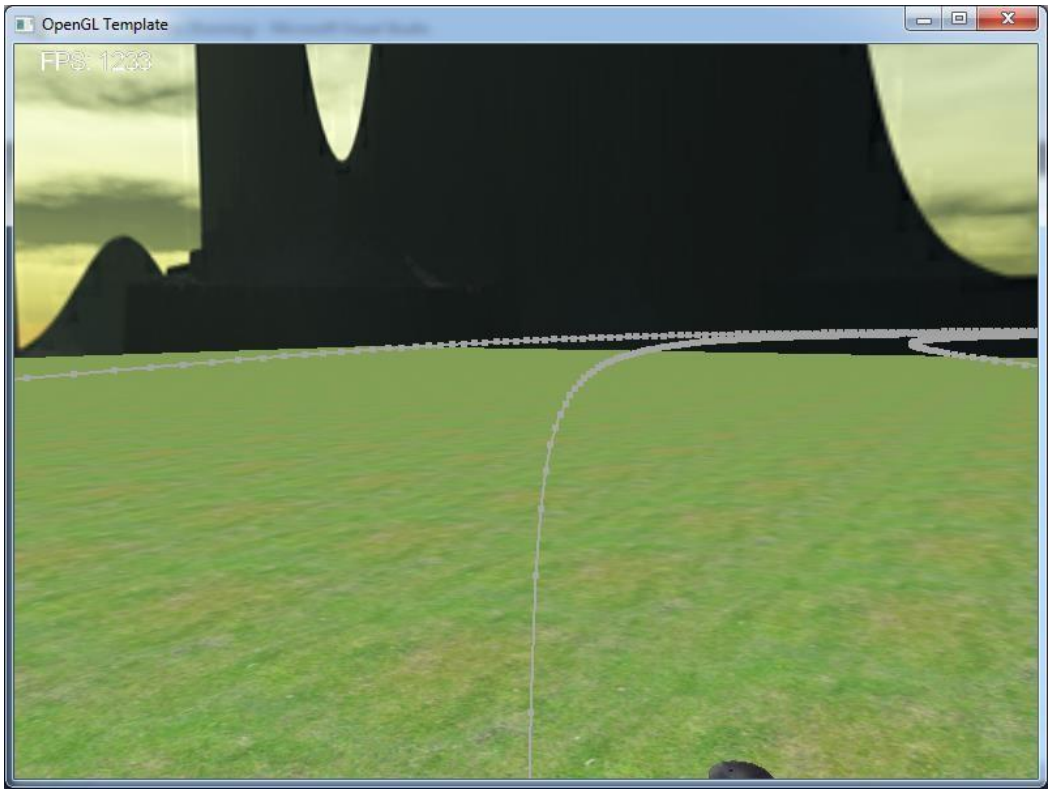
In the method `CCatmullRom:: CreateOffsetCurves()`, use a for loop to iterate over all the centreline points. At each centreline point, recompute the TNB frame, and then generate left and right offset curves, by storing points in `m_leftOffsetPoints` and `m_rightOffsetPoints` that are member variables in the class. In pseudocode, it would look something like this:

for (each centreline point **p**) determine **p**Next, the next point on the centreline
compute the **TNB** frame, based on **p** and **p**Next on the centreline and the **y** axis.

**l** = **p** - (w/2) * **N**

**r** = **p** + (w/2) * **N**
store **l** store
**r**
end for

For this, it is probably most convenient to determine **p** and **p**Next using the `m_centrelinePoints` vector, which has these points already computed. When determining **p**Next, be sure not to go outside the `m_centrelinePoints` vector at the last point on the centreline (this can be prevented using a mod (%) operation for example). Using your knowledge of VAOs / VBOs, put the offset curve points onto the GPU using two VAOs called `m_vaoLeftOffsetCurve` and `m_vaoRightOffsetCurve` already in the `CCatmullRom` class. Then in `Game::Initialise()`, call `m_pCatmullRom-> CreateOffsetCurves();` after the call to `m_pCatmullRom->CreateCentreline()`. Then in `Game::Render()`, call `m_pCatmullRom-> RenderOffsetCurves();` and write the code to render these points using a GL_LINE_STRIP primitive. I rendered mine both as GL_LINE_STRIPs and GL_POINTS. Hopefully you see something like this, moving the camera position up 5 units along the **B** axis when calling `m_pCamera->Set()` in `Game::Update()`.




## Part 3b: Connecting the vertices

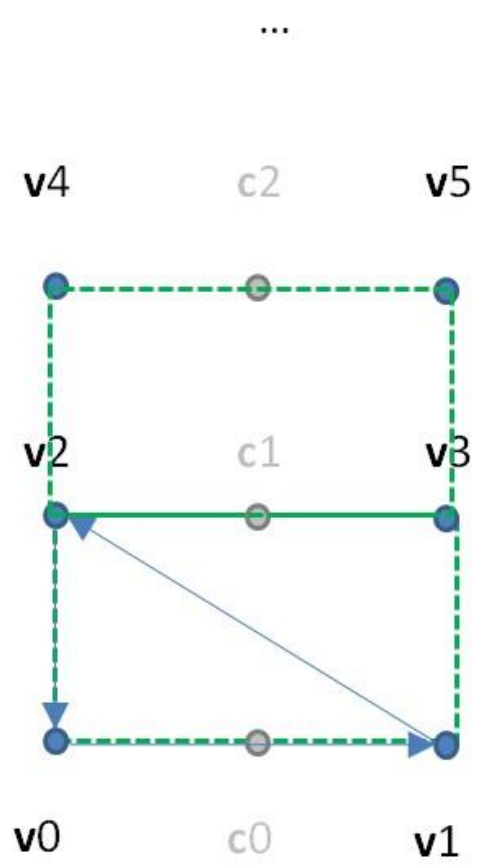
Ok, we're getting there now! We have a centreline and two offset curves defining the path edges. Next, we would like to form triangles to connect the points on the left and right offset curves to form triangles (with a consistent winding) that model the path.

Consider the figure below from a top view of the path. The points on the left offset curve are denoted as **v**0, **v**2, **v**4, and so on, whilst the points on the right offset curve are denoted as **v**1, **v**3, **v**5, and so on.

You can recognise that vertices **v**0, **v**1, **v**2, and **v**3 form a quad, and there is another quad just above formed from **v**2, **v**3, **v**4, and **v**5, and so on. Whilst this type of structure can be rendered using a GL_TRIANGLES primitive, it is more efficient to render it as a GL_TRIANGLE_STRIP

primitive. Rendering a triangle strip formed of vertices **v**0, **v**1, **v**2, **v**3, **v**4, … will form triangles as [**v**0, **v**1, **v**2], [**v**2, **v**1, **v**3], [**v**2, **v**3, **v**4], and so on.

Note we already have all the vertices specified in Part 3a and in the std::vectors `m_leftOffsetPoints` and `m_rightOffsetPoints`. So it is matter of putting the points onto a VBO in the specified order, and rendering the points as a GL_TRIANGLE_STRIP. See if you can implement this code.




Triangulating between two offset curves (left and right). The quads forming the path are rendered in green. Rendering a triangle strip formed of vertices **v**0, **v**1, **v**2, **v**3, **v**4, … will form triangles as [**v**0, **v**1, **v**2], [**v**2, **v**1, **v**3], [**v**2, **v**3, **v**4], and so on. Verify to yourself the triangles have a consistent winding.

One thing to be careful of is the last quad. We want the path to loop, so that the end connects to the beginning. For this, it will be necessary to generate two additional points in the VBO, which repeat the first two points. See if you can determine how to handle this appropriately.

In your `CCatmullRom::GeneratePath()`, write the code to form the triangles and create a VBO as described above for the path. Be sure to keep path of the number of vertices in the member variable `m_vertexCount` – this will be needed when we render. Then, in `Game::Initialise()`, call `m_pCatmullRom->CreatePath()`. In `Game::Render()`, call the `m_pCatmullRom->RenderPath()` method. Since we don't have texture mapping or lighting worked out yet, it might be best to

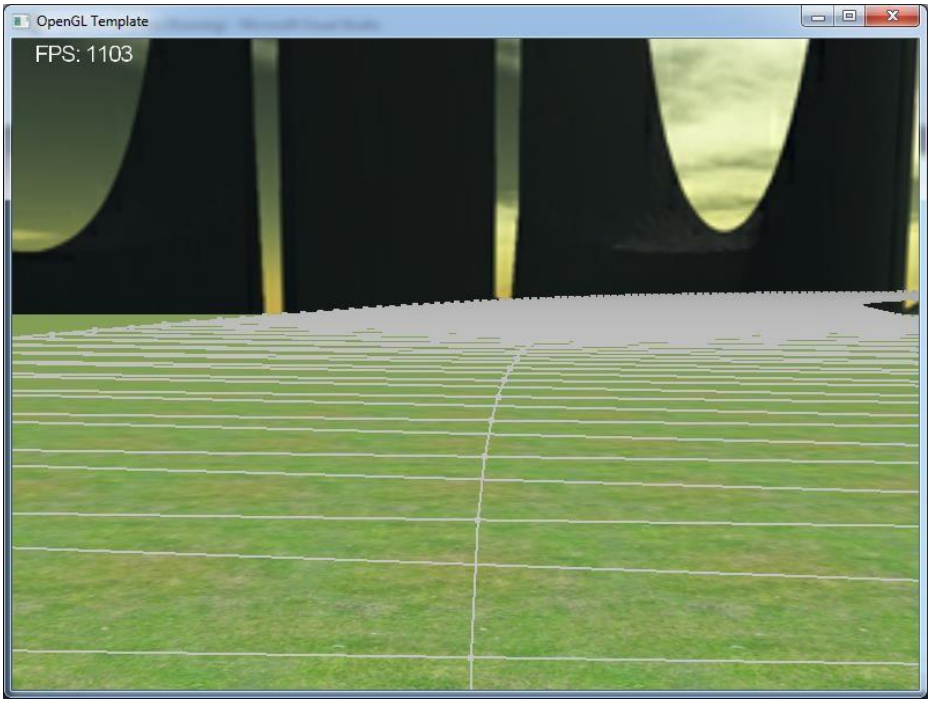
render the path as a wireframe model, by setting glPolygonMode to GL_LINE – this will show the edges of each triangle but not fill the triangles. It may also be helpful for now to disable GL_CULL_FACE for rendering the path, so that it is visible on either side.

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
```

Place these calls at the top of your implementation of `CCatmullRom::RenderPath()`. After you call `glDrawArrays()`, (or `glDrawElements()` if you're using an indexed VBO) call

```
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
```

to restore the filled polygons for the rest of the scene (skybox, terrain, etc.). If everything works, you should see something that looks like this:




Pretty awesome! Later when we discuss texture mapping and vertex normals you'll learn how to specify correct normal and texture coordinates for the path so that it is rendered with filled triangles. For now, we'll leave the path modelling as is.

## Part 4: Rotating the camera along the path

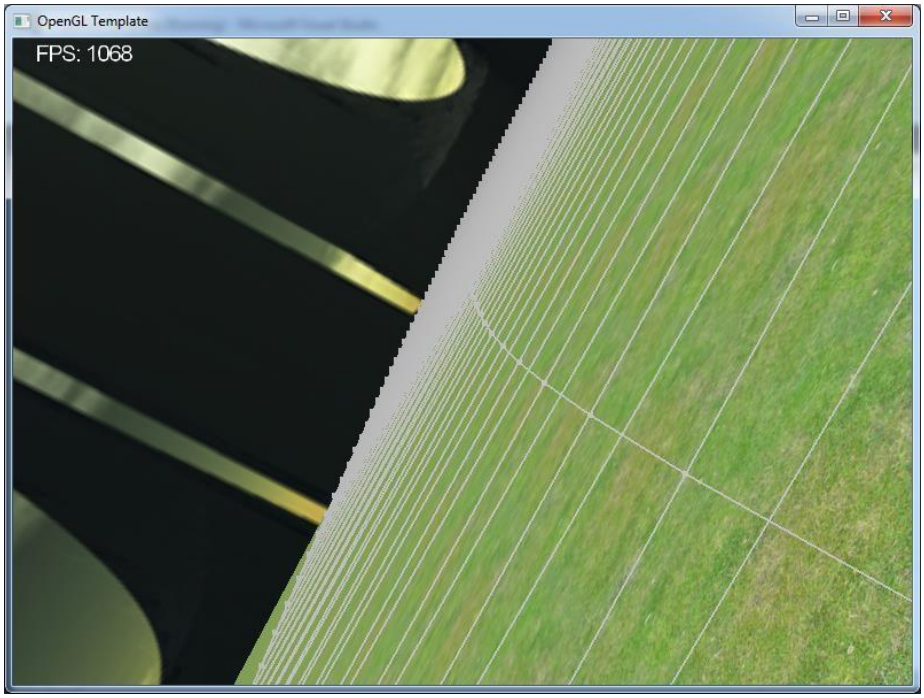
The camera is currently set to have its up vector along the **y** axis. However, in your game you *might* want to have the camera rotate around the centreline in the path (Why? Well it depends on your game, maybe this could be an interesting feature). In the game class, add a new member variable `m_cameraRotation` and in the Game constructor, initialise this variable to 0.0f.

Following the interactivity we implemented in Lab 1, increment this rotation angle when the user presses the right arrow key, for example by an amount m_dt*0.1f, and decrement by the same amount when the user presses the left arrow key.

Then, in `Game::Update()`, rotate the default up vector (**y** axis) around the tangent vector based on this angle, like this:

```
glm::vec3 up = glm::rotate(glm::vec3(0, 1, 0), m_cameraRotation, T);
```

Use this new up vector in your call to set the camera. Run your program. When you press the arrow keys, the camera should rotate!




Note there's much more that can be done here. In particular, we'll definitely want to render the triangles filled, with texture mapping and lighting – we'll cover these topics in Lectures 5 and 6.

The path of the path should be different and more interesting than the one used in this guide. Later, you may wish to include a mesh, like a car or spaceship that represents the player, and get it to move with the camera. You could refactor the code so the changes to the camera are processed in the camera class. You can implement different views as the player moves along the path.

If you're modelling walls on the edges of your path, you could again use triangle strips. It may be more convenient to use an indexed VBO, (also known as an IBO). We discussed these in class. In terms of parameters, the use of 500 samples along the centreline is a bit arbitrary, However, if the number of samples gets too small, the path may look irregular, especially at places of higher curvature. When specifying a new path, be sure that your centreline does not have excessive curvature, otherwise the path may self-intersect.

Finally, the `CCatmullRom` class includes a method `CurrentLap` that will return the current lap based on the (positive) distance the player has moved along the centreline.